

 TRAINING GUIDE — SIMPLE LANGUAGE

RadiNova AI

How the Models are Trained

A plain-English walkthrough — from downloading the dataset from Kaggle all the way to a fully trained AI model. No PhD required.



DenseNet-121 (PyTorch)



Kaggle Datasets



Transfer Learning



Medical Imaging AI

**Academic / Research Purpose Only**

This project is a hackathon prototype — NOT a certified medical device. Never use AI predictions as a substitute for real doctor diagnosis.

**Table of Contents**

- 1 What is RadiNova AI doing? (Big Picture)
- 2 What is a "Model" and How Does It Learn?
- 3 The DenseNet-121 Architecture (Simply Explained)
- 4 Transfer Learning — The Smart Shortcut
- 5 The Full Training Pipeline — Step by Step
- 6 How to Do It Manually (Complete Walkthrough)
- 7 Understanding the Results (Metrics)
- 8 Grad-CAM — Making the AI Explain Itself
- 9 Quick Reference Cheat Sheet

SECTION 01

The big picture — before any code.

RadiNova AI is a medical imaging assistant. You give it an X-ray image (like a chest X-ray), and it tells you whether the patient has **Pneumonia** or looks **Normal**. It also highlights *which region* of the X-ray made it suspicious (called Grad-CAM heatmap).

The Two Trained AI Models in This Project

- **Chest X-Ray Model** — Detects Pneumonia vs Normal. Dataset: Kaggle `paultimothymooney/chest-xray-pneumonia`
- **Limb Fracture Model** — Detects bone fractures vs normal limbs. Dataset: Kaggle `limb fracture dataset`

Other things (Blood Tests, MRI, ECG) use an LLM (like ChatGPT) with templates — those are NOT trained from scratch.

SIMPLE ANALOGY

Think of the AI like a student preparing for an exam. You show it thousands of X-rays and say "this one has pneumonia, this one is normal." After seeing enough examples, it learns the pattern and can judge new X-rays on its own — just like how you learn to recognize a cat after seeing enough cats.

SECTION 02

The fundamentals — no math needed.

An AI "model" is basically a giant math function with millions of knobs (called **weights**). At the start, all these knobs are set to random values. Training means *slowly adjusting those knobs* until the model gives correct answers.

1

Feed an X-ray image into the model. It spits out a number — e.g. "70% chance it's Pneumonia." At the start, this is basically random guessing.

2

We compare the model's answer to the real answer. The gap between them is called the **Loss**. Big loss = very wrong. Small loss = close to correct.

3

The computer figures out which knobs caused the mistake and by how much. This is called **backpropagation** — it's just math working backwards through the network.

4

The optimizer (this project uses **AdamW**) nudges every knob slightly to reduce the loss. The size of each nudge is called the **Learning Rate** (set to 0.0001 in this project).

5

Do steps 1–4 for every image in the dataset. One complete pass through all images = **1 Epoch**. This project trains for **10 epochs**. After each epoch, the model gets smarter.

✿ SIMPLE ANALOGY

Imagine you're a chef learning a new recipe. You cook it, taste it (forward pass), notice it's too salty (loss), figure out you added too much salt (backprop), add less salt next time (optimizer update). Repeat 1000 times until it's perfect. That's training.

SECTION 03

What kind of AI brain does this project use?

RadiNova uses **DenseNet-121** — a specific design of neural network that is very good at looking at images. "Dense" means every layer in the network is connected to every other layer — so no information gets lost as the image passes through.

```
# How an image passes through DenseNet-121
Input X-Ray Image (224×224 pixels, RGB) ↓ DenseBlock 1 → finds simple edges & lines ↓ DenseBlock 2 → finds shapes & textures ↓ DenseBlock 3 → finds organ shapes ↓ DenseBlock 4 → finds disease patterns ← Grad-CAM watches this layer ↓ Classifier Head → Dropout(30%) + Linear Layer ↓ Output → [NORMAL score, PNEUMONIA score]
```

By the Numbers

- **7 million+** parameters (knobs/weights) inside DenseNet-121
- **224×224 px** — image is resized to this standard size before feeding
- **2 output classes** — NORMAL (0) or PNEUMONIA (1)
- **denseblock4** — the specific layer used for Grad-CAM visual explanation

SECTION 04

Why this project doesn't train from zero.

Training DenseNet-121 from scratch on millions of images takes **weeks** and expensive GPUs. Instead, RadiNova uses **Transfer Learning** — a technique where you take a model that was already trained on 1.2 million general photos (ImageNet) and *adapt it* for X-rays.

✓ THE KEY INSIGHT

A model that already knows how to detect edges, shapes, and textures in photos can **reuse that knowledge** for medical images. You only need to teach it the final step: "this pattern = pneumonia."

This project trains in **2 stages** — a smart approach to avoid breaking the pre-learned knowledge:

⚡ Stage 1 — Warm Up (2 Epochs)

The *entire backbone* (DenseBlocks 1–4) is **frozen/locked**. Only the final classifier head is trained. This is like teaching a transferred expert just the final decision-making step.

```
# In code – freezing the backbone: for param in
model.features.parameters(): param.requires_grad = False # frozen =
can't change optimizer = AdamW(model.classifier.parameters(), lr=1e-
3) # only classifier updates
```

🔬 Stage 2 — Fine-Tune (10 Epochs)

Now **denseblock4 + norm5 + classifier** are unfrozen and fine-tuned together with a much lower learning rate (10x smaller). This lets the model specialize its deep feature detectors for medical imaging without forgetting general knowledge.

```
# Unfreezing denseblock4 for fine-tuning: for name, param in
model.features.named_parameters(): if "denseblock4" in name or
"norm5" in name: param.requires_grad = True # now trainable optimizer
= AdamW(trainable_params, lr=1e-4) # 10x smaller LR scheduler =
CosineAnnealingLR(optimizer, T_max=10) # gradually reduce LR
```

✿ SIMPLE ANALOGY

Imagine hiring a doctor who already knows general medicine (Stage 1: just teach them the radiology paperwork). Then over time, let them specialize deeper into radiology (Stage 2: fine-tune their clinical eye). Much faster than training a doctor from kindergarten.

SECTION 05

Everything that happens behind the scenes, in order.



The raw chest X-ray images are downloaded from Kaggle. The official dataset has **~14,054 images** split into train/val/test folders — but the original validation set has only 16 images which is too small to be useful.

~5,856 NORMAL images

~8,198 PNEUMONIA images



The script `datasets/prep_chest_xray.py` pools ALL images together and creates a new proper 80/10/10 split.

Why Re-Split?

The original Kaggle val set has only 16 images — statistically useless. So all 14k images are merged and re-split deterministically:

80% = Train (~11,243 images)

10% = Validation (~1,405 images)

10% = Test (~1,406 images)

Seed = 42 ensures the split is always identical (reproducible). Output is saved to `datasets/chest_xray_manifest.csv`.



Every image is transformed before being fed to the model. Training images get extra random variations (augmentation) to help the model generalize.

Training Transforms (Data Augmentation)

- Resize to 224×224 pixels
- Random Horizontal Flip (50% chance) — mirror the image
- Random Rotation (± 10 degrees) — slight tilt
- Color Jitter — slight brightness/contrast changes
- Convert to Tensor + Normalize to ImageNet mean/std

Validation/Test Transforms: Only resize + normalize. No random changes — we want consistent evaluation.



There are more PNEUMONIA images (~8,198) than NORMAL (~5,856). Without correction, the model would get lazy and predict PNEUMONIA for everything.

⚠ CLASS WEIGHTING FIX

The loss function is given class weights — NORMAL images count more per sample so the model pays equal attention to both classes despite the imbalance. Formula: `weight = total_samples / (2 × class_count)`



The two-stage training runs. After each epoch, the model is evaluated on the validation set. If it's the best so far (highest F1 score), the weights are saved to `model/weights/chest_densenet121.pth`



The saved best model is loaded and evaluated on the unseen test set (10% of all data). This gives honest performance numbers. The key metric is **Recall/Sensitivity** — because missing a real pneumonia case (false negative) is much worse than a false alarm.

SECTION 06

Step-by-step: from Kaggle download to trained model.

REQUIREMENTS

Python 3.10+, pip, a Kaggle account (free), and ideally a GPU. If no GPU, use Google Colab (free T4 GPU). Time estimate: ~30–60 min on GPU.

```
# Create a virtual environment (so packages don't conflict) python -m venv
radinova-env # Activate it # Windows: radinova-env\Scripts\activate #
Mac/Linux: source radinova-env/bin/activate # Install required packages pip
install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu118 pip install pandas numpy pillow
kagglehub scikit-learn matplotlib
```

How to Get kaggle.json

1. Go to **kaggle.com** → Log in → Click your profile photo → **Settings**

2. Scroll to "API" section → Click "**Create New Token**"
3. A file called `kaggle.json` will download
4. Place it at: `C:\Users\YourName\.kaggle\kaggle.json` (Windows)
5. Or: `~/.kaggle/kaggle.json` (Mac/Linux)

The chest X-ray dataset is ~2GB. You can download it two ways:

Method 1: Using kagglehub (Automatic — Easiest)

```
# This is what prep_chest_xray.py does automatically python -c
"import kagglehub;
kagglehub.dataset_download('paultimothymooney/chest-xray-pneumonia')"
```

It downloads to a cache folder automatically.

Method 2: Manual Download (Backup)

1. Go to: kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia
2. Click **Download** (you need to be logged in)
3. Unzip the file. You'll get a folder called `chest_xray/`
4. Place it inside: `Radionova/datasets/chest_xray/`

The folder structure should look like:

```
datasets/ └─ chest_xray/ └─ train/ | └─ NORMAL/ ← normal X-ray
images | └─ PNEUMONIA/ ← pneumonia X-ray images └─ val/ | └─
NORMAL/ | └─ PNEUMONIA/ └─ test/ └─ NORMAL/ └─ PNEUMONIA/
```

This script scans the images and creates a manifest CSV with the 80/10/10 split.

```
# Run from the Radionova/ root folder cd d:\anti gravity\Radionova # Automatic
(tries Kaggle download if folder missing) python datasets/prep_chest_xray.py #
OR if you manually placed the folder: python datasets/prep_chest_xray.py --
data-dir datasets/chest_xray
```

✅ WHAT YOU'LL SEE

```
[NORMAL] Total: 5856 → Train: 4684 (80.0%), Val: 585 (10.0%), Test: 587
(10.0%)
[PNEUMONIA] Total: 8198 → Train: 6558 (80.0%), Val: 819 (10.0%), Test: 821
(10.0%)
Manifest written to: datasets/chest_xray_manifest.csv (14054 total
records)
```

```
# Basic training with defaults (10 epochs, batch size 16, lr=0.0001) python
model/train_chest.py # Custom settings: python model/train_chest.py \ --epochs
15 \ --batch-size 32 \ --lr 0.0001 \ --output-weights
model/weights/chest_densenet121.pth
```

🔥 What Happens During Training (Per Epoch)

- Images are loaded in **batches of 16** from the manifest CSV
- Each batch goes through the model → loss computed → backprop → weights updated
- Every 25 batches, progress is printed to terminal
- After all training batches: validation set is evaluated
- If val F1-score improved: best model weights saved to .pth file

Terminal output looks like:

```
[Stage 2] Epoch [05/10] Train Loss: 0.2341 Acc: 91.2% Rec: 94.1% |
Val Loss: 0.2108 Acc: 92.8% Rec: 94.8% F1: 0.9238 >>> Best model
```

```
saved (Val F1: 0.9238) → model/weights/chest_densenet121.pth
```

After training completes, the best saved model is tested on the unseen test set:

```
===== FINAL TEST SPLIT EVALUATION
(Unseen Data) ===== Test
Accuracy: 92.40% Test Precision: 90.10% Test Recall / Sens: 94.80% ← CRITICAL
MEDICAL METRIC Test Specificity: 88.60% Test F1-Score: 0.9238 Confusion Matrix
(Test Set): Predicted NORMAL Predicted PNEUMONIA Actual NORMAL TN FP Actual
PNEUMONIA FN TP =====
```

```
# Prepare limb dataset python datasets/prep_limb_fracture.py # Train limb
model python model/train_limb.py
```

Using Google Colab Instead (Free GPU)

If you don't have a local GPU:

1. Open `model/notebooks/RadiNova_Chest_Training_Colab.ipynb`
2. Upload to Google Colab: **colab.research.google.com**
3. Runtime → Change runtime type → **T4 GPU**
4. Run all cells (Ctrl+F9)
5. Download `chest_densenet121.pth` from the Colab file browser
6. Place it in `model/weights/chest_densenet121.pth`

What do all those percentage numbers actually mean?

92.4%

Accuracy

94.8%

Recall ★

90.1%

Precision

88.6%

Specificity

0.9238

F1 Score

📖 What Each Metric Means (Plain English)

- **Accuracy (92.4%)** Out of 100 X-rays, 92 were classified correctly overall.
- ★ **Recall / Sensitivity (94.8%)** **Most important!** Out of 100 actual pneumonia patients, the AI correctly caught 94.8. Only 5.2% were missed (false negatives). Missing sick patients is dangerous, so this is maximized.
- **Precision (90.1%)** When the AI says "pneumonia", it's correct 90.1% of the time. The other 9.9% are false alarms.
- **Specificity (88.6%)** Out of 100 actually NORMAL patients, the AI correctly identifies 88.6% as normal.
- **F1 Score (0.9238)** The harmonic average of Precision + Recall. Scale: 0 (terrible) to 1 (perfect). 0.92 = very good.

✳️ ANALOGY: AIRPORT SECURITY SCANNER

Recall = catches all the actual threats (critical — don't want to miss a bomb). Precision = doesn't falsely flag innocent items too often (efficiency). In medicine, Recall (catching

all sick patients) is the top priority.

SECTION 08

How does the heatmap overlay work?

After the model makes a prediction, RadiNova generates a **heatmap** overlaid on the X-ray. Red/hot regions = areas the AI was most focused on when making its decision. This is done with a technique called **Grad-CAM**.

🔥 How Grad-CAM Works (Simply)

1. After making a prediction, ask: *"Which features in denseblock4 were most responsible?"*
2. Calculate gradients (rate of change) of the final answer with respect to each feature map
3. Average those gradients → get importance weights per feature map
4. Multiply feature maps by their importance weights → sum them up
5. Apply ReLU (keep only positive contributions) → resize to image size
6. Blend as a color heatmap (Jet colormap: red = most important) on top of the original X-ray

✖ SIMPLE ANALOGY

Imagine asking a doctor: "WHY did you say pneumonia?" They might circle the bottom-right lung region with a pen. Grad-CAM does the same thing — automatically, mathematically — by tracing which pixels most activated the "pneumonia" decision.

SECTION 09

Everything important on one page.

Complete Manual Training Commands (In Order)

```
# 1. Go to project folder cd "d:\anti gravity\Radionova" # 2.  
Activate Python environment radinova-env\Scripts\activate # 3.  
Prepare Chest X-Ray Dataset python datasets/prep_chest_xray.py # 4.  
Train Chest Model python model/train_chest.py --epochs 10 --batch-size 16 # 5. Prepare Limb Fracture Dataset python datasets/prep_limb_fracture.py # 6. Train Limb Model python model/train_limb.py --epochs 10 --batch-size 16 # 7. Train Gatekeeper (modality detector) python model/train_gatekeeper.py # 8. Start the Backend python -m uvicorn backend.main:app --port 8000 --reload
```

Key Files

- `datasets/prep_chest_xray.py` — Download + split data
- `datasets/chest_xray_manifest.csv` — Generated split index
- `model/base_classifier.py` — DenseNet-121 definition
- `model/train_chest.py` — Main training script
- `model/weights/chest_densenet121.pth` — Saved model
- `model/gradcam.py` — Heatmap generation

Key Hyperparameters

- Architecture: **DenseNet-121**
- Image size: **224 × 224 px**
- Batch size: **16**
- Stage 1 LR: **1e-3 (0.001)**
- Stage 2 LR: **1e-4 (0.0001)**
- Optimizer: **AdamW**
- Scheduler: **CosineAnnealingLR**
- Epochs: **2 (warmup) + 10 (finetune)**
- Random seed: **42**

 THE GOLDEN RULE OF THIS PROJECT

Recall / Sensitivity is the #1 metric. In medical screening, it's far worse to tell a sick patient "you're fine" (false negative) than to tell a healthy patient "you might be sick" (false positive). The model is specifically optimized to minimize false negatives — this is why class weights and recall-focused evaluation are used throughout.



RadiNova AI — Training Guide

For Academic & Research Purposes Only · Not a Medical Device

Print this page (Ctrl+P) and set background graphics ON for best PDF output